

Transformer-Architektur – Vertiefende Aspekte für Lehrkräfte und interessierte Lernende

Zielgruppe: Lehrerinnen und Lehrer sowie interessierte Schülerinnen und Schüler der gymnasialen Oberstufe

Voraussetzung: Lektüre der Primärdokumentation „Die Transformer-Architektur“

Mathematische Voraussetzungen: Lineare Algebra (Matrizen, Vektoren), Analysis (Ableitungen, Kettenregel), Grundlagen der Stochastik (Wahrscheinlichkeiten, Normalisierung)

Inhaltsverzeichnis

1. Mathematische Grundlagen: Lineare Algebra im Transformer
 2. Self-Attention: Die vollständige mathematische Herleitung
 3. Warum der Skalierungsfaktor $\sqrt{d}$ wichtig ist
 4. Positional Encoding: Sinus-Kosinus-Kodierung verstehen
 5. Backpropagation durch Attention-Schichten
 6. Layer Normalization vs. Batch Normalization
 7. Skalierungsgesetze: Warum größere Modelle besser sind
 8. Attention-Varianten und Effizienzverbesserungen
 9. Tokenisierung: Byte-Pair Encoding (BPE) im Detail
 10. RLHF: Wie Sprachmodelle auf menschliche Werte ausgerichtet werden
 11. Grenzen und offene Probleme
 12. Didaktische Hinweise für den Unterricht
 13. Literatur und Ressourcen
-

1. Mathematische Grundlagen: Lineare Algebra im Transformer

Vektoren und Matrizen als Träger von Information

Im Transformer ist **alles ein Vektor**. Ein Token der Länge d ist ein Vektor in $\mathbb{R}^d$. Die gesamte Sequenz mit n Tokens wird als Matrix $\mathbf{X} \in \mathbb{R}^{(n \times d)}$ dargestellt – jede Zeile ist ein Token-Vektor.

$$\mathbf{X} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \end{bmatrix}$$

← Token 1 (z.B. "Die")
← Token 2 (z.B. "Katze")
← Token 3 (z.B. "schläft")
...

Alle Transformationen im Transformer sind im Wesentlichen **lineare Abbildungen** (Matrizenmultiplikationen), gefolgt von nicht-linearen Aktivierungsfunktionen.

Lineare Projektion

Eine lineare Projektion transformiert einen Vektor in einen anderen Raum:

$$y = x \cdot W + b$$

Dabei ist:

- $x \in \mathbb{R}^{(1 \times d_{in})}$ - Eingabevektor (Zeilenvektor)
- $W \in \mathbb{R}^{(d_{in} \times d_{out})}$ - Gewichtsmatrix
- $b \in \mathbb{R}^{(1 \times d_{out})}$ - Bias-Vektor
- $y \in \mathbb{R}^{(1 \times d_{out})}$ - Ausgabevektor

Im Transformer werden Query, Key und Value durch solche linearen Projektionen erzeugt.

2. Self-Attention: Die vollständige mathematische Herleitung

Die Formel

Die kanonische Scaled Dot-Product Attention lautet:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Dabei sind:

- $Q \in \mathbb{R}^{(n \times d_k)}$ - Query-Matrix (eine Zeile pro Token)
- $K \in \mathbb{R}^{(n \times d_k)}$ - Key-Matrix
- $V \in \mathbb{R}^{(n \times d_v)}$ - Value-Matrix
- d_k - Dimension des Key/Query-Raums (Skalierungsfaktor)

Schritt-für-Schritt

Schritt 1: Projektionen berechnen

$$\begin{aligned} Q &= X \cdot W_Q & W_Q &\in \mathbb{R}^{(d \times d_k)} \\ K &= X \cdot W_K & W_K &\in \mathbb{R}^{(d \times d_k)} \\ V &= X \cdot W_V & W_V &\in \mathbb{R}^{(d \times d_v)} \end{aligned}$$

Die Gewichtsmatrizen W_Q, W_K, W_V werden während des Trainings gelernt.

Schritt 2: Attention-Scores berechnen

$$S = Q \cdot K^T \quad S \in \mathbb{R}^{(n \times n)}$$

$S[i][j]$ gibt an, wie ähnlich Token i (Query) und Token j (Key) sind.

Schritt 3: Skalieren und Normalisieren

$$A = \text{softmax}(S / \sqrt{d_k}) \quad A \in \mathbb{R}^{(n \times n)}$$

Jede Zeile von A summiert zu 1 $\rightarrow$ Wahrscheinlichkeitsverteilung über alle Tokens.

Schritt 4: Gewichtete Summe

$$Z = A \cdot V \quad Z \in \mathbb{R}^{(n \times d_v)}$$

$Z[i]$ ist ein gewichteter Durchschnitt aller Value-Vektoren, gewichtet nach den Attention-Scores von Token i .

Komplettes numerisches Beispiel ($n=3, d_k=d_v=2$)

Tokens: ["Die", "Katze", "schläft"]

```
X = [[1.0, 0.5],   ← "Die"
      [0.8, 1.2],   ← "Katze"
      [0.3, 0.9]]   ← "schläft"
```

Annahme: $W_Q = W_K = W_V = I$ (Einheitsmatrix, zur Vereinfachung)
 $\rightarrow Q = K = V = X$

$$S = Q \cdot K^T = \begin{bmatrix} 1.0 \cdot 1.0 + 0.5 \cdot 0.5 & 1.0 \cdot 0.8 + 0.5 \cdot 1.2 & 1.0 \cdot 0.3 + 0.5 \cdot 0.9 \\ 0.8 \cdot 1.0 + 1.2 \cdot 0.5 & 0.8 \cdot 0.8 + 1.2 \cdot 1.2 & 0.8 \cdot 0.3 + 1.2 \cdot 0.9 \\ 0.3 \cdot 1.0 + 0.9 \cdot 0.5 & 0.3 \cdot 0.8 + 0.9 \cdot 1.2 & 0.3 \cdot 0.3 + 0.9 \cdot 0.9 \end{bmatrix}$$

$$= \begin{bmatrix} 1.25 & 1.40 & 0.75 \\ 1.40 & 2.08 & 1.32 \\ 0.75 & 1.32 & 0.90 \end{bmatrix}$$

$$S / \sqrt{2} \approx \begin{bmatrix} 0.88 & 0.99 & 0.53 \\ 0.99 & 1.47 & 0.93 \\ 0.53 & 0.93 & 0.64 \end{bmatrix}$$

Softmax (zeilenweise, hier für Zeile 1):

$$\exp(0.88)=2.41, \exp(0.99)=2.69, \exp(0.53)=1.70 \rightarrow \text{Summe}=6.80$$

$$A[0] \approx [0.35, 0.40, 0.25]$$

Interpretation: "Die" schaut am stärksten auf "Katze" (0.40), dann auf sich selbst (0.35)

3. Warum der Skalierungsfaktor $\sqrt{d}$ wichtig ist

Das Problem ohne Skalierung

Wenn d_k groß ist (z. B. 64 oder 512), werden die Skalarprodukte $Q \cdot K^T$ sehr groß. Das liegt daran, dass das Skalarprodukt zweier zufälliger Vektoren der Dimension d im Erwartungswert eine Varianz von d hat:

$$\text{Var}(q \cdot k) = \sum \text{Var}(q_i \cdot k_i) = d \cdot \text{Var}(q_i) \cdot \text{Var}(k_i)$$

Bei $d=512$ können Scores in der Größenordnung von ± 20 oder mehr liegen.

Was passiert dann bei Softmax?

$$\text{softmax}([20, 0, -20]) = [\approx 1, \approx 0, \approx 0]$$

Das Softmax-Ergebnis wird **extrem konzentriert** auf den größten Wert – eine Art "Winner-takes-all"-Situation. Das Gradient durch den Softmax-Bereich nahe 0 ist verschwindend klein → **Vanishing Gradients** beim Training.

Die Lösung: Division durch $\sqrt{d_k}$

Durch die Division werden die Scores in einen Bereich gebracht, wo Softmax noch sinnvolle Gradienten liefert:

$$\text{Scores} / \sqrt{d_k} \rightarrow \text{ungefähr normalverteilte Werte} \in [-3, 3]$$

Dies ist eine einfache, aber äußerst effektive Regularisierungsmaßnahme.

4. Positional Encoding: Sinus-Kosinus-Kodierung verstehen

Warum sinusförmige Funktionen?

Die im Originalpaper verwendeten Formeln:

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})}) \end{aligned}$$

Eigenschaft 1: Eindeutigkeit

Jede Position pos erzeugt einen einzigartigen Vektor – kein zwei Positionen haben das gleiche

Encoding.

Eigenschaft 2: Glatte Interpolation

Da Sinus und Kosinus stetig sind, sind benachbarte Positionen einander ähnlich.

Eigenschaft 3: Relative Positionen kodierbar

Eine relative Verschiebung um k Positionen entspricht einer linearen Transformation:

$$\text{PE}(\text{pos} + k) = M_k \cdot \text{PE}(\text{pos})$$

für eine feste Matrix M_k (unabhängig von pos). Das erlaubt dem Modell, **relative Abstände** zwischen Tokens zu lernen.

Eigenschaft 4: Generalisierung auf ungesehene Längen

Da die Funktion analytisch definiert ist, kann das Modell auch auf Sequenzen generalisieren, die länger sind als im Training gesehen.

Moderne Alternative: Rotary Position Embedding (RoPE)

Neuere Modelle (LLaMA, Mistral, Claude) verwenden **RoPE**, das Positionsinformation direkt in die Query- und Key-Vektoren einbettet, bevor das Skalarprodukt gebildet wird:

$$\text{Attention-Score}(m, n) = \text{Re}[(R_m \cdot q)^H \cdot (R_n \cdot k)]$$

RoPE ist besonders effektiv für **Long-Context-Modelle** (sehr lange Eingabetexte).

5. Backpropagation durch Attention-Schichten

Der Gradient durch Softmax

Die Ableitung des Softmax lautet:

$$\partial \text{softmax}(z)_i / \partial z_j = \text{softmax}(z)_i \cdot (\delta_{ij} - \text{softmax}(z)_j)$$

Dabei ist δ_{ij} das Kronecker-Delta. In Matrixform:

$$J_{\text{softmax}} = \text{diag}(a) - a \cdot a^T$$

wobei $a = \text{softmax}(z)$.

Der Gradient durch das Skalarprodukt

$$\begin{aligned} \partial(Q \cdot K^T) / \partial Q &= K \\ \partial(Q \cdot K^T) / \partial K &= Q \end{aligned}$$

Der Gradient fließt **symmetrisch** durch Query und Key.

Residual Connections erleichtern den Gradientenfluss

Ohne Residual Connections muss der Gradient durch jede Schicht propagieren und wird mit jeder Multiplikation kleiner (Vanishing Gradient). Mit der Residual Connection:

$$\begin{aligned}\text{output} &= \text{LayerNorm}(x + F(x)) \\ \partial \text{output} / \partial x &= I + \partial F(x) / \partial x\end{aligned}$$

Der Gradient enthält die Einheitsmatrix (I) → er kann immer ungedämpft direkt zu den früheren Schichten fließen, unabhängig davon, wie klein $\partial F(x) / \partial x$ ist.

6. Layer Normalization vs. Batch Normalization

Batch Normalization (für CNNs typisch)

Normalisiert über die **Batch-Dimension**:

$$\text{BN}(x) = (x - \mu_{\text{batch}}) / \sigma_{\text{batch}} \cdot \gamma + \beta$$

Problem bei Transformers: Textsequenzen haben variable Längen; Batches enthalten unterschiedlich lange Sätze. Die Batch-Statistiken sind daher instabil.

Layer Normalization (Standard im Transformer)

Normalisiert über die **Feature-Dimension** (innerhalb eines einzigen Vektors):

$$\text{LN}(x) = (x - \mu_x) / \sigma_x \cdot \gamma + \beta$$

Dabei sind μ_x und σ_x Mittelwert und Standardabweichung der Elemente von x . γ und β sind lernbare Parameter (eine Skalierung und Verschiebung pro Dimension).

Vorteil: Unabhängig von der Batch-Größe und Sequenzlänge – ideal für Transformer.

Pre-LN vs. Post-LN

Das Originalpaper verwendet **Post-LN** (Normalisierung nach der Sublayer-Ausgabe). Neuere Modelle bevorzugen oft **Pre-LN** (Normalisierung vor der Sublayer-Eingabe), da dies stabiler zu trainieren ist:

$$\begin{aligned}\text{Post-LN: } \text{output} &= \text{LayerNorm}(x + \text{Sublayer}(x)) \\ \text{Pre-LN: } \text{output} &= x + \text{Sublayer}(\text{LayerNorm}(x))\end{aligned}$$

7. Skalierungsgesetze: Warum größere Modelle besser sind

Empirische Skalierungsgesetze (Kaplan et al., 2020)

Forscher bei OpenAI stellten fest, dass die Performance eines Sprachmodells vorhersagbar von drei Faktoren abhängt:

$$L(N, D, C) \approx A/N^{\alpha_N} + B/D^{\alpha_D} + C_{irr}$$

- **N** – Anzahl der Parameter (Modellgröße)
- **D** – Menge der Trainingsdaten (Tokens)
- **C** – Trainingsrechenaufwand (FLOPs)
- $\alpha_N \approx 0.076, \alpha_D \approx 0.095$ (empirisch ermittelt)

Kernaussage: Wenn das Rechenbudget verdoppelt wird, ist es meist besser, sowohl das Modell als auch die Datenmenge proportional zu vergrößern, statt nur eines von beiden.

Chinchilla-Gesetze (Hoffmann et al., 2022)

Deepmind verfeinerte die Skalierungsgesetze: Für optimale Performance sollte die Anzahl der Trainings-Tokens **20× die Anzahl der Parameter** betragen:

$$\text{Optimale Tokens} \approx 20 \times N_{\text{Parameter}}$$

Beispiel: Ein Modell mit 70 Milliarden Parametern sollte auf ca. 1.4 Billionen Tokens trainiert werden.

Diese Erkenntnis führte dazu, dass LLaMA und ähnliche Modelle kleinere, aber länger trainierte Architekturen verwenden.

8. Attention-Varianten und Effizienzverbesserungen

Das Skalierungsproblem der Standard-Attention

Standard-Attention hat **quadratische Komplexität** in der Sequenzlänge n:

$$\begin{aligned} \text{Zeitkomplexität:} & \quad O(n^2 \cdot d) \\ \text{Speicherkomplexität:} & \quad O(n^2) \end{aligned}$$

Für eine Sequenz von 100.000 Tokens ist die Attention-Matrix $100.000 \times 100.000 = 10$ Milliarden Einträge – das passt nicht mehr in den GPU-Speicher.

Flash Attention (Dao et al., 2022)

Flash Attention berechnet das gleiche Ergebnis wie Standard-Attention, aber deutlich effizienter

durch geschicktes Tiling:

- Speicher: **$O(n)$** statt $O(n^2)$
- Geringere Datenübertragung zwischen GPU-SRAM und HBM
- Bis zu **7× schneller** in der Praxis, bei identischem Ergebnis

Grouped-Query Attention (GQA)

In Multi-Head Attention hat jeder Kopf eigene Q-, K-, V-Projektionen. Bei GQA teilen sich mehrere Query-Köpfe ein Key-Value-Paar:

Standard MHA: h Köpfe, h Key-Vektoren, h Value-Vektoren

GQA: h Köpfe, h/g Key-Vektoren, h/g Value-Vektoren (g = Gruppen)

Vorteil: Deutlich weniger Speicher beim Inference (besonders im KV-Cache), kaum Qualitätsverlust.

Sparse Attention

Statt alle n^2 Paare zu berechnen, werden nur die k wichtigsten Verbindungen beachtet:

BigBird, Longformer: Lokale Fenster + globale Tokens

→ $O(n)$ Komplexität statt $O(n^2)$

9. Tokenisierung: Byte-Pair Encoding (BPE) im Detail

Das Problem mit naiver Wort-Tokenisierung

Ein Vokabular aus ganzen Wörtern hätte Millionen Einträge. Unbekannte Wörter (z. B. Eigennamen, Neologismen) könnten nicht repräsentiert werden.

Byte-Pair Encoding (BPE) - Algorithmus

BPE ist ein kompressionsbasierter Algorithmus (ursprünglich von Gage 1994):

Algorithmus:

1. Starte mit Zeichenvokabular: $\{"a", "b", \dots, "z", " ", \dots\}$
2. Zähle alle benachbarten Symbol-Paare im Corpus
3. Füge das häufigste Paar als neues Symbol zusammen
4. Wiederhole Schritt 2-3, bis Vokabulargröße = Ziel (z.B. 50.000)

Beispiel-Iteration:

Corpus: "low low lower lowest"

Tokens: [l,o,w,_] [l,o,w,_] [l,o,w,e,r,_] [l,o,w,e,s,t,_]

Häufigstes Paar: (l, o) $\rightarrow$ 4x

$\rightarrow$ Merge: (l, o) = "lo"

Nächste Iteration:

Häufigstes Paar: (lo, w) $\rightarrow$ 4x

$\rightarrow$ Merge: "low"

Usw. bis Vokabulargrenze erreicht

SentencePiece und Unigram Language Model

Modernere Tokenizer (z. B. für LLaMA, T5) verwenden **SentencePiece mit Unigram LM**: Sie wählen das Vokabular so, dass die Wahrscheinlichkeit des Corpus maximiert wird. Das ist prinzipiell flexibler als BPE.

Byte-Level BPE (GPT-2, GPT-4)

OpenAI's Modelle tokenisieren direkt auf Byte-Ebene (UTF-8 Bytes), was jede Zeichenkodierung unterstützt und keine explizite Behandlung unbekannter Zeichen erfordert.

10. RLHF: Wie Sprachmodelle auf menschliche Werte ausgerichtet werden

Das Grundproblem

Ein auf Next-Token-Prediction trainiertes Modell kann Texte erzeugen, die statistisch plausibel sind, aber:

- Faktisch falsch sind
- Schädlich oder beleidigend sind
- Nicht den eigentlichen Intentionen des Nutzers entsprechen

Reinforcement Learning from Human Feedback (RLHF)

RLHF wurde von Christiano et al. (2017) eingeführt und für Sprachmodelle von InstructGPT (Ouyang et al., 2022) popularisiert. Der Prozess hat drei Phasen:

Phase 1: Supervised Fine-Tuning (SFT)

Trainingsdaten: Menschlich verfasste Beispiel-Antworten auf Anfragen

Ziel: Modell lernt, Anweisungen zu folgen

Phase 2: Reward Model Training

Menschliche Bewerter: Vergleichen je 2 Modellantworten auf dieselbe Anfrage
→ "Antwort A ist besser als Antwort B"
Reward Model: Lernt, die Qualität einer Antwort vorherzusagen
→ Gibt einen skalaren "Qualitätsscore" aus

Phase 3: PPO-Training (Proximal Policy Optimization)

Policy (= Sprachmodell) erzeugt Antwort
Reward Model bewertet Antwort → Skalar r
PPO-Update: Erhöhe Wahrscheinlichkeit guter Antworten, senke schlechte
KL-Regularisierung: Verhindert zu starke Abweichung vom SFT-Modell

Die Verlustfunktion beim PPO:

$$L = E[r(x, y)] - \beta \cdot KL[\pi_{\theta}(y|x) || \pi_{SFT}(y|x)]$$

Constitutional AI (Anthropic)

Anthropic (Entwickler von Claude) verwendet eine Variante namens Constitutional AI (CAI), bei der das Modell seine eigenen Antworten anhand eines Regelwerks (der „Verfassung“) bewertet und verbessert – weniger abhängig von menschlichem Feedback für jedes einzelne Beispiel.

Direct Preference Optimization (DPO)

DPO (Rafailov et al., 2023) ist eine neuere Alternative zu RLHF, die das Reward Model eliminiert und direkt aus Präferenzpaaren trainiert:

$$L_{DPO} = -E[\log \sigma(\beta \cdot \log \pi_{\theta}(y_w|x)/\pi_{ref}(y_w|x) - \beta \cdot \log \pi_{\theta}(y_l|x)/\pi_{ref}(y_l|x))]$$

Dabei ist y_w die bevorzugte und y_l die weniger bevorzugte Antwort. DPO ist stabiler und einfacher zu implementieren als RLHF mit PPO.

11. Grenzen und offene Probleme

Kontextfenster und Langzeitgedächtnis

Transformer haben ein begrenztes **Kontextfenster** (die maximale Anzahl von Tokens, die sie gleichzeitig verarbeiten können). Neuere Modelle haben Kontextfenster von 128.000 bis über 1.000.000 Tokens, aber die quadratische Komplexität der Attention bleibt ein Problem.

Offene Frage: Wie können Transformer echtes Langzeitgedächtnis entwickeln, das über einzelne Sitzungen hinausgeht?

Halluzinationen

Transformer generieren Text basierend auf statistischen Mustern – sie haben kein Weltmodell und keine Möglichkeit zu unterscheiden, ob eine Aussage faktisch wahr ist. Dies führt zum Problem der **Halluzinationen**: das Modell erfindet überzeugend klingende, aber falsche Fakten.

Lösungsansätze: Retrieval-Augmented Generation (RAG), verbesserte RLHF-Methoden, Tool Use.

Reasoning und Mathematik

Transformer sind gut in Pattern-Matching, haben aber Schwierigkeiten mit:

- Komplexer mehrschrittiger Logik
- Exakter Arithmetik
- Kombinatorischen Problemen

Chain-of-Thought Prompting (Wei et al., 2022) hilft dabei erheblich: Modelle werden trainiert, Lösungsschritte explizit zu verbalisieren, bevor sie eine Antwort geben.

Kausalität vs. Korrelation

Transformer lernen korrelative Muster aus Text. Das bedeutet:

- Sie können sagen, dass Regen und nasse Straßen oft zusammen vorkommen
- Aber sie "verstehen" nicht im kausalen Sinne, dass Regen die Straße nass macht

Echtes kausales Schlussfolgern bleibt eine offene Herausforderung.

Energieverbrauch und Nachhaltigkeit

Das Training großer Transformer-Modelle verbraucht enorme Mengen an Strom. Das Training von GPT-3 wurde auf ca. **1.300 MWh** geschätzt, was einem CO₂-Äquivalent von mehreren Hundert Tonnen entspricht.

12. Didaktische Hinweise für den Unterricht

Unterrichtsvorschlag: Stufenweiser Aufbau

Unterrichtseinheit 1 (2h): Von Wörtern zu Vektoren

- Einstieg: Wortsemantik und Ähnlichkeit
- Experiment mit Word2Vec-Demo (Online-Tool)
- Handschriftliche Berechnung: kleines Embedding-Beispiel

Unterrichtseinheit 2 (2h): Self-Attention greifbar machen

- Rollenspiel: Schüler sind Tokens, tauschen Informationszettel aus
- Handrechnung der Attention-Formel (3 Tokens, 2 Dimensionen)
- Visualisierung mit BertViz

Unterrichtseinheit 3 (2h): Transformer in der Praxis

- Vergleich verschiedener Modelle (BERT vs. GPT)
- Diskussion: Chancen und Risiken großer Sprachmodelle
- Tokenizer-Experimente mit eigenen Texten

Häufige Missverständnisse bei Lernenden

Missverständnis	Richtigstellung
„Der Transformer liest Wörter wie ein Mensch“	Er verarbeitet alle Tokens gleichzeitig, nicht sequenziell
„Attention bedeutet, das Modell versteht die Bedeutung“	Attention ist eine mathematische Gewichtung, kein Verstehen im menschlichen Sinn
„Größere Modelle wissen immer mehr“	Größere Modelle generalisieren besser, können aber immer noch halluzinieren
„Das Modell erinnert sich an frühere Gespräche“	Ohne explizites Gedächtnis gibt es kein Langzeitgedächtnis zwischen Sitzungen
„GPT denkt nach“	Das Modell wählt das nächste Token probabilistisch – es gibt keine symbolische Schlussfolgerung

Differenzierung

Für leistungsstarke Schüler: Lineare Algebra im Kontext von Attention; Implementierung einer simplen Attention-Schicht in Python/NumPy

Für breite Klassen: Schwerpunkt auf konzeptueller Ebene, Analogien und Visualisierungen

Fächerübergreifend:

- *Mathematik:* Vektoren, Matrizen, Wahrscheinlichkeitsrechnung, Optimierung
- *Ethik/Sozialkunde:* KI-Regulierung, Bias in Trainingsdaten, Transparenz
- *Deutsch/Sprache:* Sprachmodelle und Sprachverständnis, philosophische Fragen

13. Literatur und Ressourcen

Originalarbeiten (Open Access)

- Vaswani et al. (2017): *Attention Is All You Need* – arxiv.org/abs/1706.03762
- Devlin et al. (2018): *BERT: Pre-training of Deep Bidirectional Transformers* – arxiv.org/abs/1810.04805
- Brown et al. (2020): *Language Models are Few-Shot Learners (GPT-3)* – arxiv.org/abs/2005.14165
- Hoffmann et al. (2022): *Training Compute-Optimal LLMs (Chinchilla)* – arxiv.org/abs/2203.15556
- Dao et al. (2022): *FlashAttention* – arxiv.org/abs/2205.14135
- Rafailov et al. (2023): *Direct Preference Optimization* – arxiv.org/abs/2305.18290

Lehrressourcen

- **The Illustrated Transformer** (Jay Alammar): jalammar.github.io/illustrated-transformer – hervorragende visuelle Erklärung
- **3Blue1Brown: "But what is a GPT?"** (YouTube) – mathematisch fundiert, visuell ansprechend
- **Andrej Karpathy: "Let's build GPT from scratch"** (YouTube) – vollständige Implementierung
- **BertViz** (Jesse Vig): Attention-Visualisierung für BERT – github.com/jessevig/bertviz
- **Hugging Face Transformers** – huggingface.co/docs/transformers – praktische Implementierungen

Bücher (für tieferen Einstieg)

- Géron, A. (2022): *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.) – Kapitel 16
- Jurafsky, D. & Martin, J. H. (2023): *Speech and Language Processing* (3rd ed., Draft) – web.stanford.edu/~jurafsky/slp3 – kostenlos online

Python-Implementierungen für den Unterricht

python

```

# Minimale Self-Attention in NumPy (zur Veranschaulichung)
import numpy as np

def softmax(x):
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e_x / e_x.sum(axis=-1, keepdims=True)

def self_attention(X, W_Q, W_K, W_V):
    """
    X:      (n, d)    - Eingabe-Sequenz
    W_Q, W_K: (d, d_k) - Projektionsmatrizen
    W_V:      (d, d_v) - Value-Projektion
    """
    Q = X @ W_Q      # (n, d_k)
    K = X @ W_K      # (n, d_k)
    V = X @ W_V      # (n, d_v)

    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k) # (n, n)
    weights = softmax(scores)        # (n, n)
    output = weights @ V             # (n, d_v)

    return output, weights

# Beispiel
np.random.seed(42)
n, d, d_k = 3, 4, 2
X = np.random.randn(n, d)
W_Q = np.random.randn(d, d_k)
W_K = np.random.randn(d, d_k)
W_V = np.random.randn(d, d_k)

output, attn_weights = self_attention(X, W_Q, W_K, W_V)
print("Attention Weights:\n", attn_weights.round(3))
print("Output Shape:", output.shape)

```

*Dokument erstellt als Ergänzung zu: → [transformer_schueler.md]
 Alle zitierten Arbeiten sind Open Access und frei zugänglich.*